Numero:_____ Nome:_____

Instituto Politécnico de Tomar

Escola Superior de Tecnologia

Cursos de Engenharia Informática

Estruturas de Dados e Algoritmos - **Prova Teórica**

Ano Letivo de 2025-2026 – 1º semestre – 1ª Frequência Duração: 2 horas

Notas: A interpretação do enunciado faz parte da resolução. Se encontrar incoerências ou ambiguidades, resolva-as da melhor maneira, e explicite as decisões tomadas. Tenha presente, além disso, que os seus programas devem tirar o melhor partido da matéria da cadeira. Apresente-os com clareza, bem formatados e limpos.

Grupo I (4 Valores)

Assinale com uma cruz (**X**) nos espaços respetivos se as afirmações seguintes são verdadeiras ou falsas.
NOTA: Cada resposta correta vale **0.25** valores e cada resposta errada vale **-0.25** valores com um mínimo de zero valores no total do grupo.

Questão	V	F
A análise da complexidade espacial permite estimar o espaço em memória que um determinado algoritmo necessita para ser executado.		
A um algoritmo com uma complexidade temporal de $O(N^2)$ chama-se um algoritmo de complexidade quadrática ou polinomial de segunda ordem.		
As seguintes complexidades temporais estão ordenadas por ordem crescente (da melhor para a pior): O(1), O(logN), O(N).		
Uma das desvantagens dos algoritmos recursivos é consumir mais memória do que a mesma versão iterativa.		
Algoritmos com complexidade temporal constante são interessantes porque o tempo de execução não depende da quantidade de dados.		
As pilhas são estruturas de dados com acesso limitado à informação e devem ser usadas como estruturas auxiliares na resolução de algoritmos.		
Para guardar a mesma informação, listas duplamente ligadas precisam de mais espaço em memória do que listas simplesmente ligadas.		
A inserção ordenada numa lista duplamente ligada tem complexidade temporal O(N).		
As pilhas têm complexidade temporal O(N) nas operações de inserção, remoção e pesquisa.		
Um algoritmo que cria N subproblemas para resolver o problema inicial tem complexidade temporal exponencial.		
O algoritmo para pesquisar um valor numa matriz quadrada tem complexidade temporal quadrática.		
A notação O representa a complexidade temporal dos algoritmos no pior caso.		
Estuda-se a complexidade temporal dos algoritmos para estimar o tempo de execução face ao volume de dados.		
A pesquisa de um valor nas listas ligadas tem complexidade temporal O(N).		
Ao implementar uma pilha sobre um array, esta pode conter um número ilimitado de elementos.		
A notação O representa a complexidade temporal dos algoritmos no melhor caso.		

Grupo II

Considere a seguinte função

```
void segredo(int[] x, int tam) {
    for (int i = 0; i < tam; i++) {
        x[i] = x[i] * 2;
    }
}

void quebraCabecas(int[] x, int tam) {
        int i;
        for (i = 0; i < tam; i++)
                System.out.print(x[i]);
        segredo(x,tam);
        for (i = 0; i < tam; i++)
                System.out.print(x[i]);
}
```

[1,0 valores] a) O que faz a função segredo função **segredo**.

[1,0 valores] b) Qual é a complexidade espacial e temporal da função **quebraCabecas**.

[1,0 valores] c) É possível melhorar a complexidade espacial da função segredo? Fundamente a sua resposta.

Grupo III

Para a seguinte sequência de números: 5, 3, 1, 7, 6, 4, 2, 9, 8

[1,0 valores] Mostre a árvore de pesquisa binária resultante da sua introdução.

[1,0 valores] Mostre o resultado da travessia da árvore em pre-order.

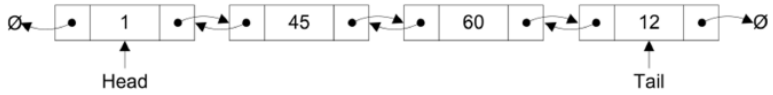
[1,0 valores] Mostre a árvore de pesquisa binária após a remoção dos valores 6, 7 e 5

Grupo IV

Sabendo que uma lista duplamente ligada é representada pelas seguintes classes:

```
class Item {
    Object value;
    Item next;
    Item previous;
}
```

```
class LDL {
    Item head;
    Item tail;
    LDL() {
        head = tail = null;
    }
    int contaOcorrencias(Object value) {
      // devolve as ocorrencias do value na lista
    }
}
```



Crie os seguintes métodos na classe LDL:

[2,0 valores] a) void inserirFim(Object value) que insere o valor passado por parâmetro no fim da lista.

[2,0 valores] b) Object removeFim() que remove o último elemento da lista e devolve-o. Se a lista estiver vazia o método devolve null.

[2,0 valores] c) int substitui(Object o1, Objecto o2) que substitui na lista todas das ocorrências do objecto o1, pelo objecto o2 e devolve o número de substituições que fez.

[2,0 valores] d) LDL removeDuplicados() que devolve uma lista sem os elementos duplicados.

Ex:

lista = {1, 4, 5, 4, 1, 2, 7, 9}

Resultado = {1, 4, 5, 2, 7, 8}

[2,0 valores] e) LDL intersecao(LDL l1, LDL l2) que retorna uma lista com o resultado da interseção entre as listas l1 e l2. Ao fazer a intersecção das listas, se existirem valores duplicados, estes não serão adicionados ao resultado:

Ex:

l1 = {1, 4, 3, 4, 1, 2, 7, 9}

l2 = {2, 3, 2, 8}

Resultado = {2, 3}